Structure of the ESS forecast-error model

What is built where, which optimisation problems are solved, and how they connect

Code base: `Code/src/` (git commit `18ef358` plus working-tree changes of 2026-10-27)

The question the code answers. Baldick's SOC-based design (ISO optimises the battery's state of charge) and the current design (battery submits a bid u_t and an offer c_t per hour, no SOC in the clearing) give the *same* prices and dispatch when the bids are built from the true clearing prices $\lambda^\star$ (Theorem 1, proven and checked numerically). The experiments measure how much social welfare is lost when the bids are instead built from a *forecast* $\hat{\lambda}$, and which properties of the error (size σ, autocorrelation ρ, bias, correlation across batteries ω) drive the loss.

Contents

1 Big picture **2**

2 Repository map and dependencies **3**

3 Where the model is created: the case dictionary **4**

4 The optimisation models **5**

4.1 M1 – Baldick's SOC design $\mathcal{P}^{\mathrm{SOC}}$ (the benchmark) . . . 6
4.2 M2 – Current design $\mathcal{P}^{\mathrm{CD}}$ and the equivalent bids (E) . . . 6
4.3 M3 – The battery's self-schedule $\mathcal{P}_e^{\mathrm{SS}}(\lambda)$, with a tie-breaker . . . 6
4.4 M4 – The multiplier polyhedron Π (Prop. 5) and the selection rule . . . 7
4.5 From multipliers to a bid curve: the quantity rule . . . 7
4.6 M5 – Day-ahead clearing of the current design (`market.clear_hour`) . . . 8
4.7 M6 – Real-time repair of the battery (`realtime.feasible_schedule`) . . . 9
4.8 M7 – Generator redispatch (`realtime.redispatch_hour`) . . . 9
4.9 Welfare and metrics . . . 10

5 The forecast error **10**

5.1 Model (`forecasts.forecast_prices`) . . . 10
5.2 Seeds: one error shape per draw, shared by every setting . . . 11
5.3 Where the error is inserted . . . 12

6 Choosing the error values, and how to defend them **12**

6.1 Forecast error size σ: $\{0, 2.5, 5, 10, 20, 40\}$ % of $\bar{\lambda}$ . . . 12
6.2 Autocorrelation ρ: $\{0, 0.5, 0.9\}$ between consecutive intervals . . . 13
6.3 Bias: $\{-10, 0, +10\}$ % of $\bar{\lambda}$ . . . 13
6.4 Common share ω: $\{0, 0.5, 1\}$ (stage 2, several batteries) . . . 13
6.5 Price floor $\underline{\lambda} = 0$ . . . 13
6.6 Not error parameters, but chosen alongside them . . . 14

7 Inside the multiplier polyhedron Π **14**

7.1 What Π is, in plain words . . . 14
7.2 At the true prices: a square of choices, all equally good . . . 14
7.3 Under forecast error: the valid ranges move and open up . . . 15

8 One experiment draw, end to end **17**
8.1 What each bid family isolates 17

9 Seeing each step: notebook, explorer and albums **18**

10 Configuration and stages (`config.py`) **18**

11 Verification layer **19**

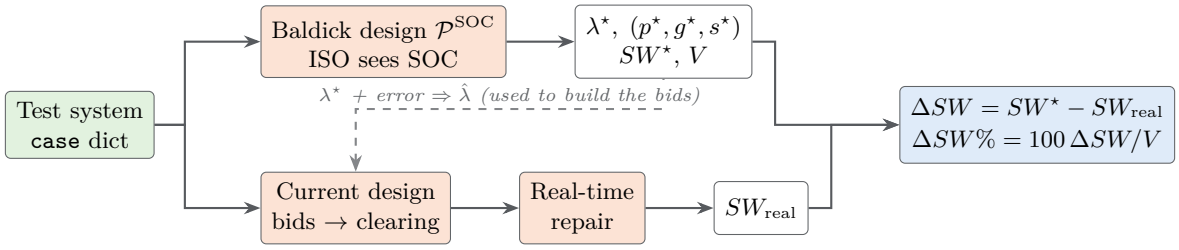
12 Numbers: benchmark and stage 0 **19**

13 Notation crosswalk **20**

14 Caveats found while reading the code **21**

1 Big picture

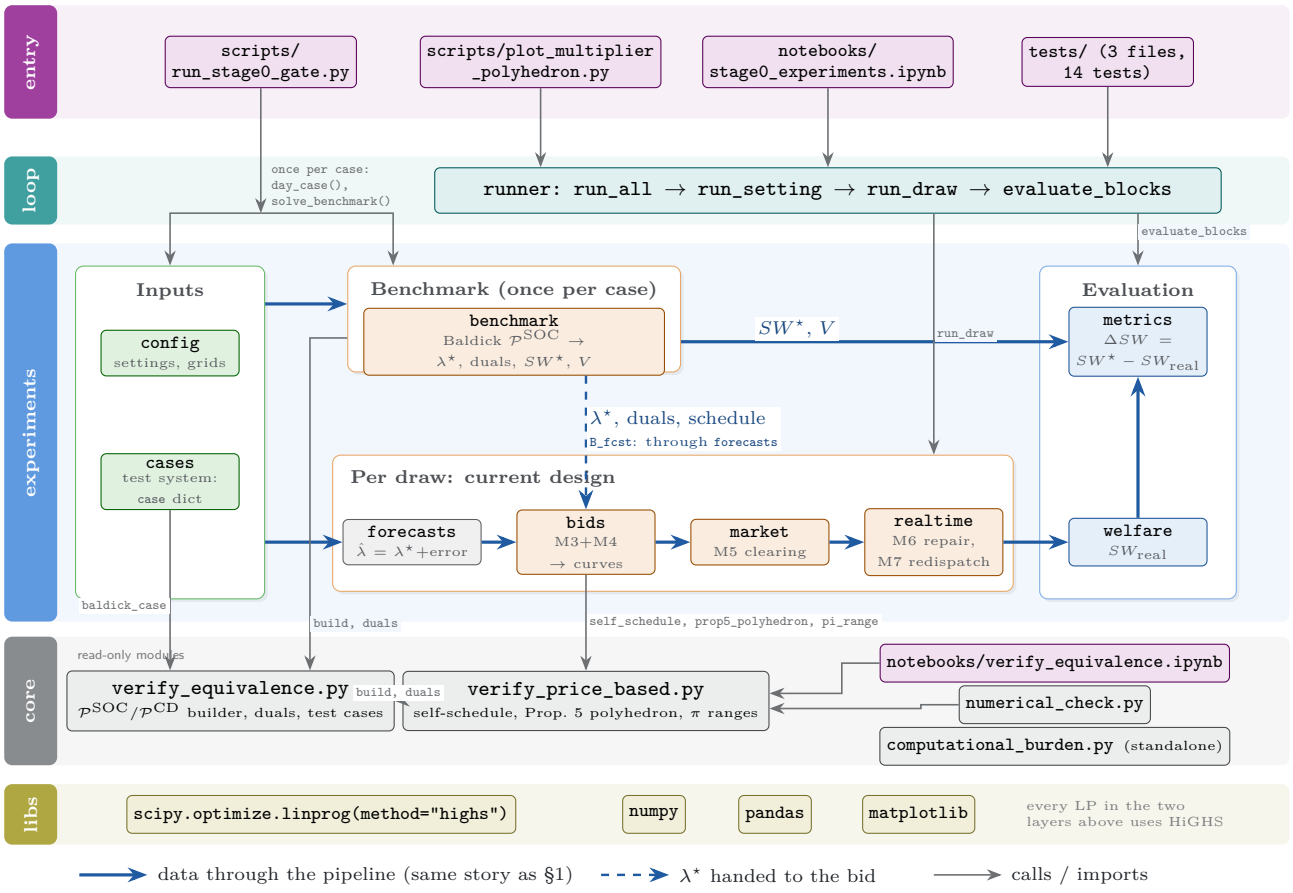
Every experiment is one comparison: a *benchmark* welfare $SW^\star$ from Baldick's design, solved once, against a *realised* welfare SW_{real} obtained by pushing a bid through the current design and then repairing whatever the battery cannot physically deliver.



Three facts to keep in mind throughout:

1. **The welfare yardstick is the same function** for the benchmark and for the realised outcome (generation cost, redispatch premium, load shedding at VOLL, battery degradation, minus the terminal value $w\,s_T$). Only the dispatch it is evaluated at changes.
2. **The error enters in one place only**: the price vector the battery uses to build its own bid (family `B_fcst`). The system data, the clearing and the welfare evaluation always use the *true* parameters.
3. **The benchmark is solved once per test system**; each experiment draw solves only the battery's own small problems, the hourly clearing and the real-time repair.

2 Repository map and dependencies

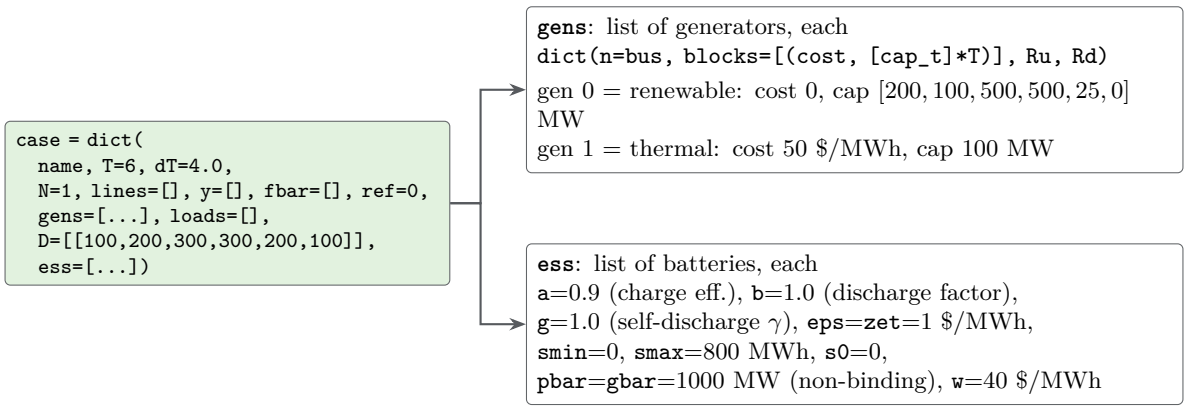


Read the experiments layer left to right, like the big picture of §1: the inputs feed the Baldick benchmark (top row, solved once by the entry script) and, per draw, the current design (bottom row): forecast, bid construction, day-ahead clearing, real-time repair. Welfare and ΔSW close the loop with $SW^\star$ and V from the benchmark. `forecasts` is used only by `B_fcst`; the other bid families take $\lambda^\star$ (or nothing, `B_priv`) directly. Thin arrows show who calls or imports whom: `runner.run_draw` drives the bottom row and `evaluate_blocks` runs everything from `market` to `welfare`. `market`, `realtime`, `welfare` and `metrics` import nothing from the repository (they receive the `case` dict). Not drawn: `forecasts` imports `config`; the polyhedron script and the stage-0 notebook also call experiment modules and `verify_price_based` directly; the verification notebook and `numerical_check` import both core modules.

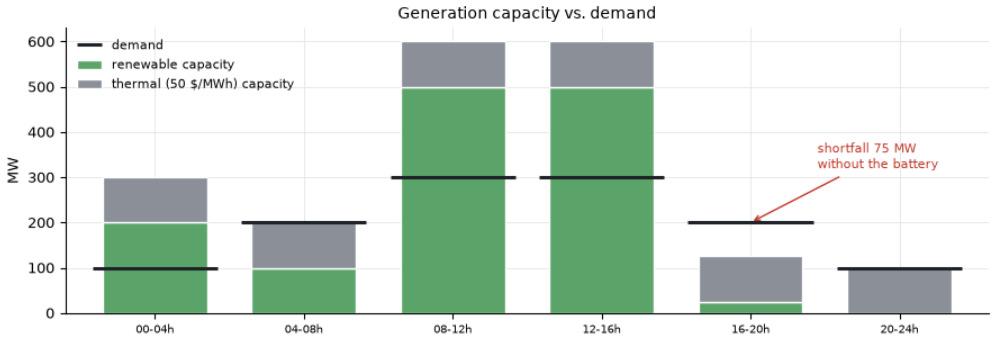
File	What it contains	Main public objects
`verify_equivalence.py`	The LP builder shared by both designs; dual extraction with the thesis sign convention; the KKT and Prop. 1–2 checks. **Read-only.**	`baldick_case`, `one_bus_case`, `LP`, `build`, `duals`, `verify`
`verify_price_based.py`	The battery's own self-schedule LP; the polyhedron of admissible multipliers (Prop. 5); ranges of π_t. **Read-only.**	`self_schedule`, `prop5_polyhedron`, `pi_range`, `dual_face_range`
`numerical_check.py`	Re-runs every proposition on Baldick's three cases and writes the thesis figures/table.	`run_case`
`computational_burden.py`	Standalone timing benchmark SOC vs. CD (own sparse builder, random instances).	`instance`, `build`, `run`
`experiments/config.py`	Frozen dataclasses for one experiment setting, and all stage grids.	`ErrorModel`, `BidRule`, `RealTime`, `RunSettings`, `stage0_settings()`, `stage1_settings()`
`experiments/cases.py`	Test systems.	`day_case`, `baldick_cases`, `fleet_case` (not implemented)
`experiments/benchmark.py`	Solves $\mathcal{P}^{\mathrm{SOC}}$ once and the no-storage system; keeps prices, duals, the battery and generator schedules.	`solve_benchmark` → `Benchmark` (`lam_star`, `pi_star`, `p/g/s_star`, `q_star`, `SW_star`, `V`)
`experiments/forecasts.py`	The AR(1) common/specific price-error model.	`forecast_prices`, `ar1_unit_variance`
`experiments/bids.py`	The four bid families and the two-block quantity rule.	`bid_star`, `bid_star_p`, `bid_fcst`, `bid_priv`, `quantity_blocks`
`experiments/market.py`	Day-ahead clearing of the current design, hour by hour.	`clear_cd`, `clear_hour`
`experiments/realtime.py`	SOC-feasible repair of the battery schedule; generator redispatch.	`feasible_schedule`, `redispatch_hour`
`experiments/welfare.py`	Realised welfare and battery profit.	`realised_welfare`, `ess_profit_day_ahead`
`experiments/metrics.py`	ΔSW, decomposition, $\sigma^\star$, summaries.	`dsw`, `decompose`, `sigma_star`, `summarize`
`experiments/runner.py`	The loop: one row per draw, one CSV per setting, `meta.json`. `evaluate_blocks` runs everything after bid construction and returns every intermediate result.	`run_draw`, `evaluate_blocks` → `Outcome` (`da`, `p/g/s_rt`, `q_rt`, `shed_rt`, `welfare`), `run_setting`, `run_all`
`scripts/run_stage0_gate.py`	Runs the stage-0 settings, writes `results/stage0/`, asserts the gate.	`main`
`scripts/plot_multiplier_polyhedron.py`	Figures of the multiplier polyhedron (§7).	`projection`, `fig_true`, `fig_forecast`, `fig_3d`
`notebooks/stage0_experiments.ipynb`	Every step shown on one draw, the interactive explorer and the albums (§9).	`simulate_draw`, `draw_panels`, `render_album`
`docs/figures/album_seed1036.html`	Standalone album page: one slider per error parameter, opens without Python.	–

3 Where the model is created: the `case` dictionary

There is no model class. A test system is a plain Python `dict`; every LP is *assembled from it on the fly* by the function that needs it. `cases.day_case()` simply returns `verify_equivalence.baldick_case(smax=800, w=40)` (the pilot's own `day_case` is not in the repository; this substitution is documented in `cases.py` and the README).



- **Time**: $T = 6$ intervals of $\Delta T = 4\,\text{h}$ (one day). Powers are MW, energies MWh, so every power term carries a factor `dT` when it enters an energy balance or a cost.
- **Renewables** are a zero-cost generator with a time-varying capacity. Curtailment is therefore implicit: the LP simply dispatches less than the cap (this is what happens at $t = 3, 4$, where $\lambda^\star = 0$).
- **Demand** is inelastic (`loads=[]`), so welfare reduces to minus total cost.
- **Load shedding** is not a variable of `build`. `benchmark.with_shedding_generator` adds a generator priced at VOLL = 1000 $/MWh (capacity 2× peak demand); `market` and `realtime` add an explicit shedding column priced at VOLL.



Generation capacity against demand (notebook §1): without storage, 16–20h is 75 MW short.

4 The optimisation models

Seven LP families are solved, all with SciPy `linprog(method="highs")`. The table locates each one; the subsections give the abstract model.

	Model	Built in	Called by	per draw
M1	$\mathcal{P}^{\text{SOC}}$ Baldick clearing	`verify_equivalence.build(case,"SOC")`	`benchmark`	once/case
M2	$\mathcal{P}^{\text{CD}}$ single-block clearing	`verify_equivalence.build(case,"CD",bids)`	verification only	–
M3	$\mathcal{P}^{\text{SS}}(\lambda)$ self-schedule	`verify_price_based.self_schedule`	`bids`	1
M4	Prop. 5 polyhedron Π: range / vertex	`verify_price_based.pi_range`, `bids._solver_vertex`	`bids`	$2T$ or 1
M5	Hourly day-ahead clearing	`market.clear_hour`	`runner`	T
M6	SOC-feasible repair (L1)	`realtime.feasible_schedule`	`runner`	1 per ESS
M7	Hourly redispatch	`realtime.redispatch_hour`	`runner`	T

4.1 M1 – Baldick's SOC design $\mathcal{P}^{\mathrm{SOC}}$ (the benchmark)

Indices: generator i, block k, battery e, bus n, interval t. In the single-bus case the flow, angle and ramp rows exist in the code but are empty or non-binding.

$$\min \sum_t \Delta T\Big[\sum_{i,k} c_{ik} q_{ikt} - \sum_{m,j} U_{mj} d_{mjt} + \sum_e (\varepsilon_e p_{et} + \zeta_e g_{et})\Big] - \sum_e w_e s_{eT}$$

s.t. $\sum d_{nt} + D_{nt} + \sum_l A_{nl} f_{lt} - \sum q_{nt} - \sum_{e\in n}(g_{et} - p_{et}) = 0$	$[\lambda_{nt}]$	(A1) balance
$f_{lt} = y_l \sum_n A_{nl}\theta_{nt}, \quad \theta_{\mathrm{ref},t} = 0$	$[\kappa, \xi]$	(A2),(A4)
$-R_i^d \le \sum_k q_{ikt} - \sum_k q_{ik,t-1} \le R_i^u$	$[\tau]$	(A6) ramps
$s_{et} = \gamma_e s_{e,t-1} + \Delta T\, a_e p_{et} - \Delta T\, b_e g_{et}$	$[\pi_{et}]$	(A8) SOC dynamics
$\gamma_e s_{e,t-1} + \Delta T\, a_e p_{et} \le \bar{s}_e$	$[\bar{\nu}_{et}]$	(A9) upper SOC
$\gamma_e s_{e,t-1} - \Delta T\, b_e g_{et} \ge \underline{s}_e$	$[\underline{\nu}_{et}]$	(A10) lower SOC
$0 \le q \le \bar{Q},\ 0 \le d \le \bar{d},\ \lvert f\rvert \le \bar{f},\ 0 \le p \le \bar{p},\ 0 \le g \le \bar{g}$		(A3,5,7,11,12)

(A9)/(A10) bound the SOC reached by charging alone and by discharging alone, which is the same as $\underline{s} \le s_t \le \bar{s}$ whenever the battery does not charge and discharge in the same interval. The constant $w_e \gamma_e^T s_{e0}$ of the terminal value is dropped in the LP and added back in `benchmark.solve_benchmark`: $SW^\star = -(f^{\mathrm{SOC}\star} + \sum_e w_e \gamma_e^T s_{e0})$.

Dual extraction (`verify_equivalence.duals`). SciPy returns $\partial f^\star/\partial \mathrm{rhs}$. With the note's Lagrangian $L = f + y(\mathrm{lhs} - \mathrm{rhs})$: equality and $\le$ rows give $y = -\mathrm{marginal}$, upper bounds $\bar{y} = -\mathrm{upper.marginal}/\Delta T$, lower bounds $\underline{y} = \mathrm{lower.marginal}/\Delta T$; power rows (tags `lam, kap, xi, tauU, tauL`) are divided by ΔT so every multiplier is in \$/MWh. The result is a dict keyed by row tag, e.g. `y["pi", e, t]`.

KKT rows used later. Stationarity in p, g, s:

$$\varepsilon + \lambda_t - a(\pi_t - \bar{\nu}_t) + \bar{\rho}_t - \underline{\rho}_t = 0, \qquad \zeta - \lambda_t + b(\pi_t + \underline{\nu}_t) + \bar{\sigma}_t - \underline{\sigma}_t = 0,$$

$$\pi_t = \gamma\,(\pi_{t+1} - \bar{\nu}_{t+1} + \underline{\nu}_{t+1}), \qquad \pi_T = w.$$

So π_t is the marginal value of stored energy, propagated backwards from the terminal value w, with a jump wherever an SOC limit binds.

4.2 M2 – Current design $\mathcal{P}^{\mathrm{CD}}$ and the equivalent bids (E)

Same LP with s and (A8)–(A10) removed; the battery's cost term becomes its bid:

$$\sum_t \Delta T\,(-u_{et}\, p_{et} + c_{et}\, g_{et}), \qquad \boxed{u_{et} = a_e(\pi_{et} - \bar{\nu}_{et}) - \varepsilon_e, \qquad c_{et} = \zeta_e + b_e(\pi_{et} + \underline{\nu}_{et})} \quad \text{(E)}$$

Substituting (E) into the KKT rows above shows that $(x^\star, \lambda^\star)$ satisfies the KKT conditions of $\mathcal{P}^{\mathrm{CD}}$: this is Theorem 1. M2 is only used by the verification scripts; the experiments use the multi-block version M5.

4.3 M3 – The battery's self-schedule $\mathcal{P}_e^{\mathrm{SS}}(\lambda)$, with a tie-breaker

The battery alone, as a price-taker against a price vector λ (true *or* forecast). Its true objective (minus its profit) is

$$f(p, g, s; \lambda) = \sum_t \Delta T\big[(\varepsilon + \lambda_t)p_t + (\zeta - \lambda_t)g_t\big] - w\, s_T,$$

and the plan the experiments use solves

$$\boxed{\min_{p,g,s}\ f(p, g, s; \lambda)\ -\ \kappa \sum_t \Delta T\,(p_t + g_t) \quad \text{s.t. (A8)–(A12) of battery } e}$$

with $\kappa = 10^{-3}$ \$/MWh (`BidRule.tie_break_reward`). Lemma 4: at $\lambda = \lambda^\star$ Baldick's schedule is optimal for f. Output: a planned schedule $(p^\circ, g^\circ, s^\circ)$, used both as the *quantity* of the bid and to define M4.

What the extra term does. It rewards throughput $\sum_t \Delta T(p_t + g_t)$, the MWh the battery trades, with κ per MWh. κ is far below any real profit difference, so it never makes the battery give up profit: it only decides between plans that earn *exactly* the same. In words: *among all plans that maximise the battery's true profit, take the one that trades the most.* The code enforces this strictly (`bids.self_schedule`): it checks that the plan's true objective f equals the optimum of f (to 10^{-9} relative) and halves κ until it does. So the plan is always

optimal for the true problem, and everything built on that (Lemma 4, the polyhedron Π of M4, Theorem 1) still holds. The term appears only here. Welfare (M7) is computed from the realised dispatch at the true costs, so κ never enters SW or ΔSW. $\kappa = 0$ switches the tie-breaker off.

Implementation: the term only changes the objective, and it equals running the unchanged self-schedule LP (`verify_price_based`) with $\varepsilon \to \varepsilon - \kappa$ and $\zeta \to \zeta - \kappa$. Those two parameters appear nowhere else in that LP, so its constraints are reused, not copied. An assert keeps $2\kappa < \varepsilon + \zeta$, so simultaneous charging and discharging can never look free.

Why we apply it. At the true prices the battery is indifferent between many plans, because $\lambda^\star$ is full of ties: a break-even round trip at 00–08h ($(43.1 + \varepsilon)/\alpha = 49 = 50 - \zeta$), two free intervals at 08–16h (both priced 0) and two evening intervals at 50. Without a rule, the plan is whichever optimal vertex the LP solver happens to return. That is an artefact of the algorithm, not a decision of the owner, and it can change with the solver, its version or the order of the constraints. The tie-breaker replaces that accident with a stated, economically sensible rule: a battery that is indifferent offers its capacity to the market rather than holding it back.

Self-schedule at $\lambda^\star$	plan p° / g° (MW, 6 intervals)	throughput	`B_star_p` loss
$\kappa = 0$ (solver's vertex)	$p^\circ = (0, 0, 0, 222, 0, 0)$, $g^\circ = (0, 0, 0, 0, 200, 0)$	1 689 MWh	6.32 % of V
$\kappa = 10^{-3}$ (tie-breaker)	$p^\circ = (222, 0, 0, 222, 0, 0)$, $g^\circ = (0, 200, 0, 0, 0, 200)$	3 378 MWh	**1.30** % of V

The solver's vertex skips the morning round trip, because at break-even it earns nothing. The tie-broken plan runs it (buy at 00–04h, sell at 04–08h), as Baldick's schedule does, so the zero-error loss is 1.30% of V instead of 6.32%.

What it does not do. (i) It breaks only *exact* ties. Any forecast error makes one plan strictly better, and the tie-breaker then has nothing to choose. It changed the plan in 0 of the 750 forecast draws of stage 0, so it moves only the $\sigma = 0$ numbers. (ii) It cannot break a *timing* tie, where two plans trade the same MWh at different times (buy the free energy at 08–12h or at 12–16h; sell at 16–20h or at 20–24h). The remaining 1.30% is exactly such a tie: the plan buys all 222 MW at 12–16h, where only 200 MW of free energy exists, while Baldick's schedule also buys at 08–12h. The battery ends up with 80 MWh less stored and 100 MWh more thermal energy is burned (+5 000 \$). Adding +400 \$ of redispatch, −800 \$ for the 20 MWh left at the end and −189 \$ of degradation gives 4 411 \$ = 1.30% of V. (iii) A larger κ that deliberately gives up profit for activity would act on *near*-ties too. That would be a behavioural assumption, not a tie-breaker, and is not used.

4.4 M4 – The multiplier polyhedron Π (Prop. 5) and the selection rule

Given λ and the planned schedule, the admissible multipliers $x = (\pi, \bar{\nu}, \underline{\nu}) \in \mathbb{R}^{3T}$ are

$\pi_T = w, \qquad \pi_t = \gamma(\pi_{t+1} - \bar{\nu}_{t+1} + \underline{\nu}_{t+1})$	(costate)
$\bar{\nu}_t \geq 0$ if (A9) is tight at t, else $\bar{\nu}_t = 0$; same for $\underline{\nu}_t$, (A10)	(compl. slackness)
$u_t \leq \lambda_t$ if $p^\circ_t = 0$, $u_t \geq \lambda_t$ if $p^\circ_t = \bar{p}$, $u_t = \lambda_t$ otherwise	(price consistency)
$c_t \geq \lambda_t$ if $g^\circ_t = 0$, $c_t \leq \lambda_t$ if $g^\circ_t = \bar{g}$, $c_t = \lambda_t$ otherwise	

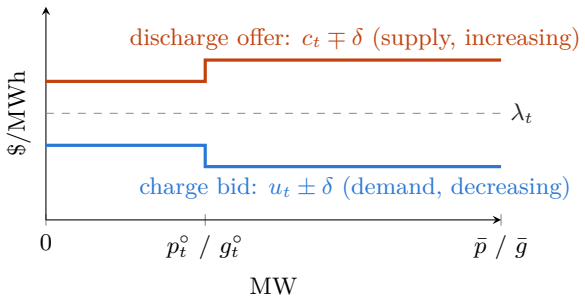
with u, c from (E). Any point of Π gives valid bids; Π is generally *not* a single point (degenerate duals). `pi_range` solves $2T$ LPs (min/max π_t) to get $[\mathrm{lo}_t, \mathrm{hi}_t]$. The point is chosen by `BidRule.rule`:

- `"prelimit"` (`bids._prelimit_vertex`): $\pi_1 = \mathrm{hi}_1$, then $\pi_t = \mathrm{clip}(\pi_{t-1}/\gamma,\ \mathrm{lo}_t,\ \mathrm{hi}_t)$; $\bar{\nu}, \underline{\nu}$ are then solved from the costate equation (only one can be non-zero at each t).
- `"solver"` (`bids._solver_vertex`): the vertex of Π minimising $\sum_t \pi_t$ (one LP).

4.5 From multipliers to a bid curve: the quantity rule

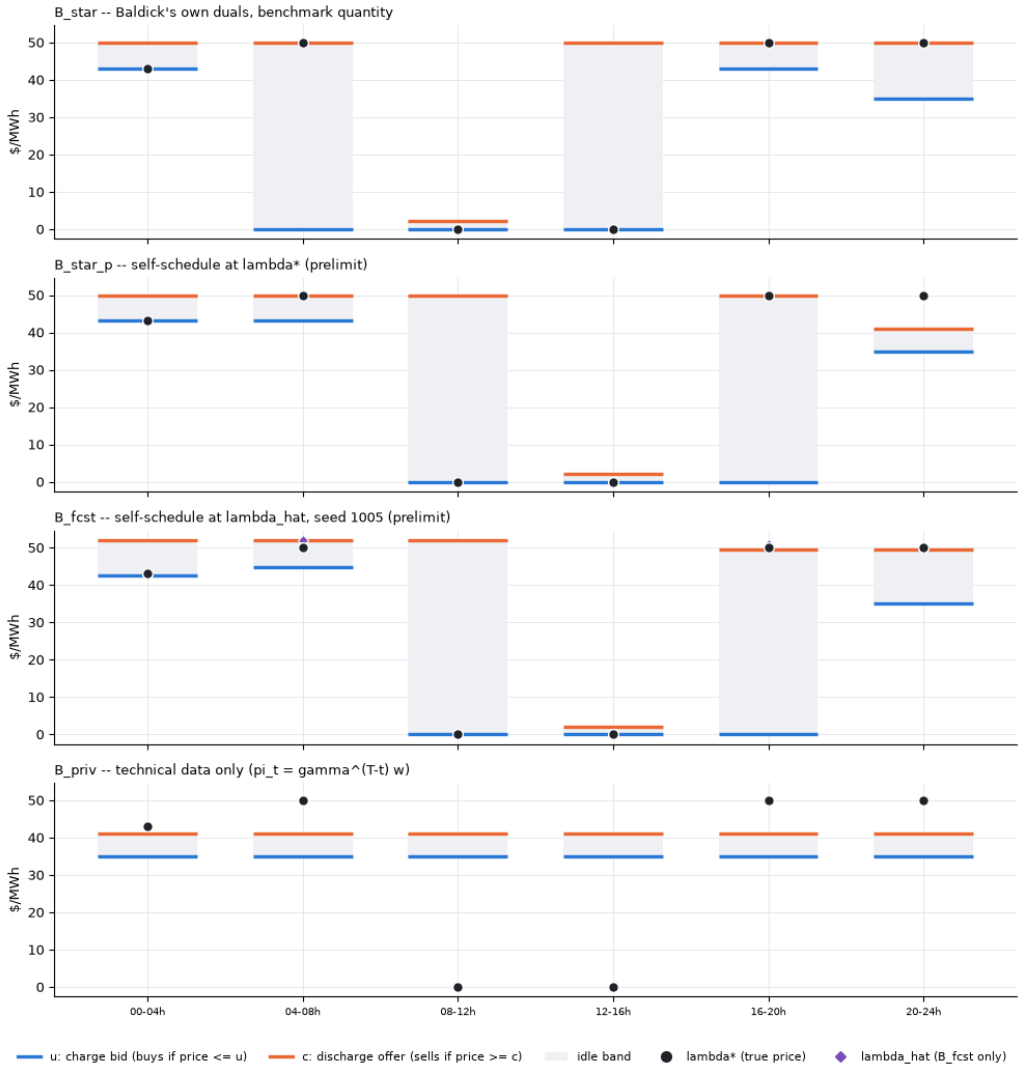
`bids.quantity_blocks` turns $(u_t, c_t, p^\circ_t, g^\circ_t)$ into two blocks per side with a small spread $\delta = 0.01$ \$/MWh, so the clearing prefers the planned quantity at the price $\lambda^\star$:

$$\text{charge: } (u_t + \delta,\ p^\circ_t),\ (u_t - \delta,\ \bar{p} - p^\circ_t); \qquad \text{discharge: } (c_t - \delta,\ g^\circ_t),\ (c_t + \delta,\ \bar{g} - g^\circ_t).$$



Illustrative prices. The band $[u_t, c_t]$ always contains $\lambda_t^\star$ (for `B_star`), with $\lambda_t^\star$ on an edge whenever the battery is marginal. `B_priv` uses one full-capacity block per side and no spread.

The actual bids of the four families on the test system (notebook §4; `B_fcst` for seed 1005, $\sigma = 1$ \$/MWh). Blue edge = charge bid u_t (buys if the price is $\leq u_t$), orange edge = offer c_t (sells if $\geq c_t$), dot = $\lambda_t^\star$:



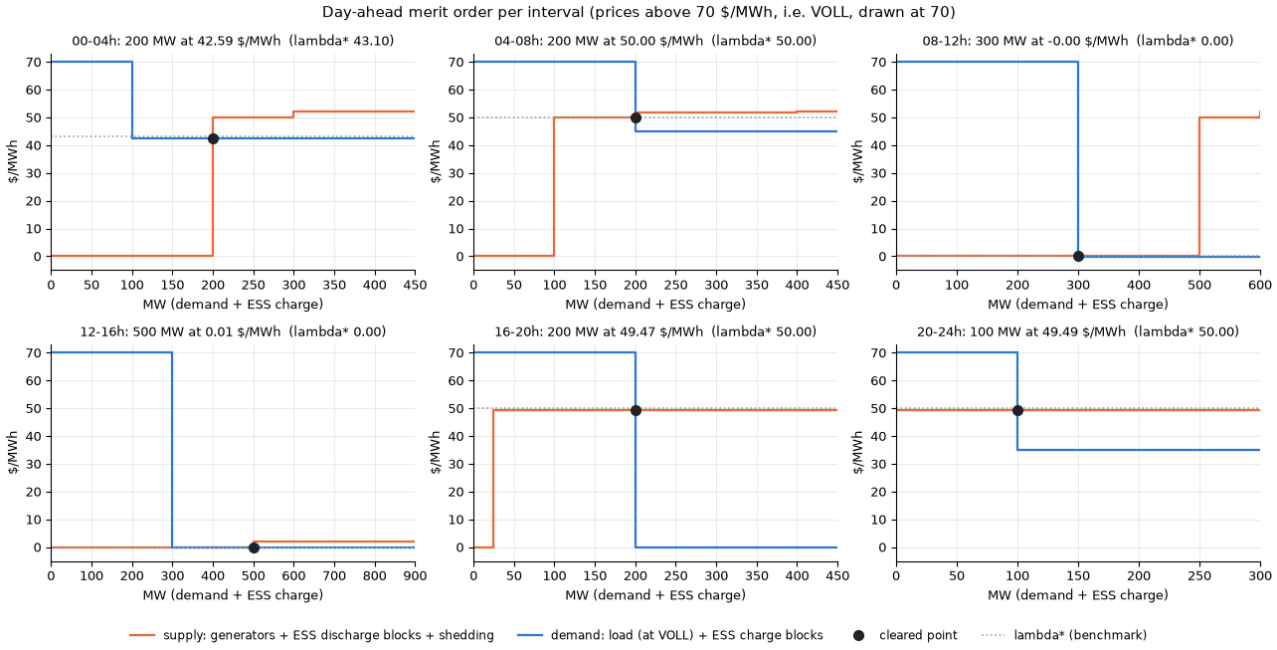
4.6 M5 – Day-ahead clearing of the current design (`market.clear_hour`)

Single bus, one LP per hour t (valid because ramps are non-binding in `day_case`). Generator blocks j at their true cost c_j, each battery's blocks k, a shedding column:

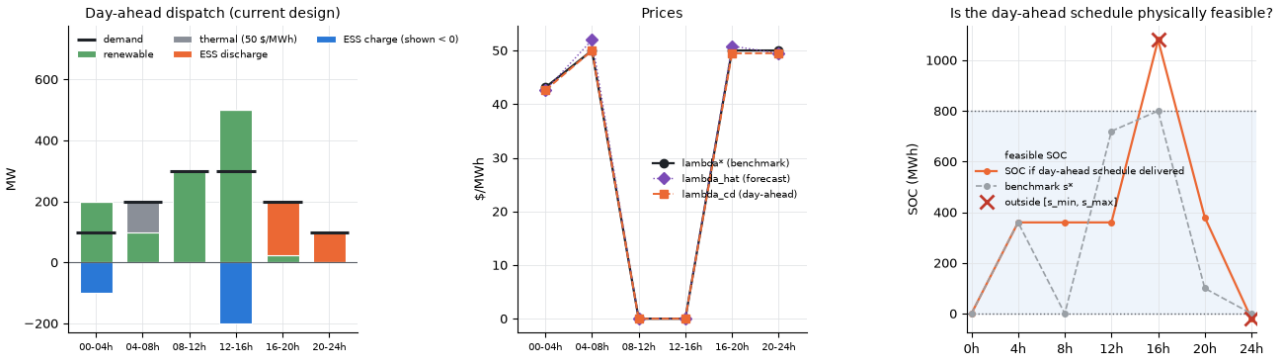
$$\min \sum_j c_j q_j - \sum_{e,k} u_{ek}\, x^p_{ek} + \sum_{e,k} c_{ek}\, x^g_{ek} + \text{VOLL} \cdot sh \quad \text{s.t.} \quad \sum_j q_j - \sum_{e,k} x^p_{ek} + \sum_{e,k} x^g_{ek} + sh = D_t \;\; [\lambda_t^{\text{CD}}]$$

with $0 \leq q_j \leq \bar{Q}_{jt}$, $0 \leq x_{ek} \leq$ block size, $0 \leq sh \leq D_t$. There is **no SOC constraint**: the result $p^{da}_e = \sum_k x^p_{ek}$, $g^{da}_e = \sum_k x^g_{ek}$ may be physically infeasible for the battery. Output: `DayAheadResult(q_da, p_da, g_da, shed, lam_cd)`.

Each interval is a merit-order auction (notebook §7.3, seed 1005, $\sigma = 1$): supply = generators + the battery's offer blocks + shedding, demand = load + the battery's bid blocks; the dot is the cleared point.



Because no SOC enters the clearing, the accepted schedule can take the battery outside its limits (right panel: the SOC the day-ahead schedule *would* produce, red × = infeasible):



4.7 M6 – Real-time repair of the battery (`realtime.feasible_schedule`)

The battery delivers the physically feasible schedule closest (L1) to what it sold, with its *true* parameters:

$$\min_{p,g,s} \sum_t |p_t - p_t^{da}| + |g_t - g_t^{da}| \quad \text{s.t. (A8)–(A12)}, \qquad \text{Undeliverable} = \Delta T \sum_t \left(|p_t - p_t^{da}| + |g_t - g_t^{da}|\right) \text{ MWh}.$$

Implemented with the split $p = p^{da} + p^+ - p^-$, $p^{\pm} \geq 0$ (same for g): $7T$ variables.

4.8 M7 – Generator redispatch (`realtime.redispatch_hour`)

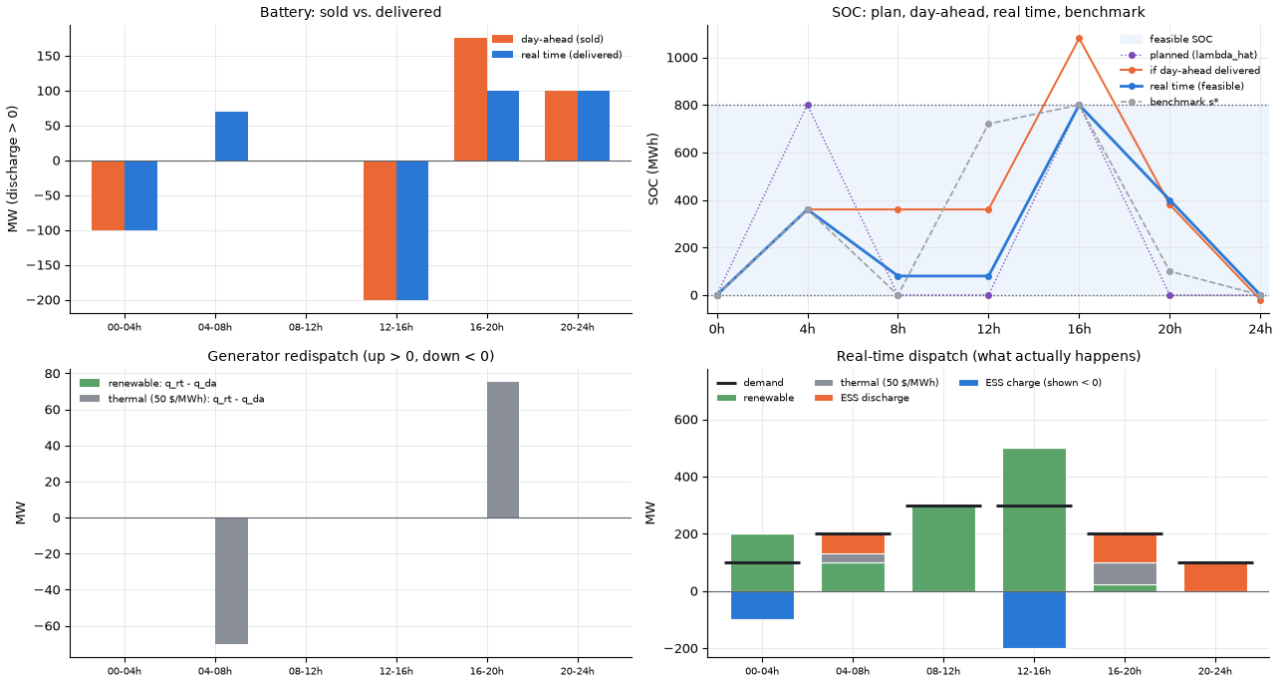
Per hour, generators move from q^{da} to cover the net demand left after the battery's real-time action $N_t = D_t - \sum_e g_{et}^{rt} + \sum_e p_{et}^{rt}$, paying a premium ψ per MWh moved in either direction:

$$\min_{q^+,q^-,sh} \sum_j (c_j + \psi) q_j^+ + (\psi - c_j) q_j^- + \text{VOLL} \cdot sh$$

$$\text{s.t.} \sum_j (q_j^+ - q_j^-) + sh = N_t - \sum_j q_j^{da}, \qquad 0 \leq q_j^+ \leq \bar{Q}_{jt} - q_j^{da}, \quad 0 \leq q_j^- \leq q_j^{da}.$$

$q^{rt} = q^{da} + q^+ - q^-$. Moving down saves c_j but still costs ψ.

M6 and M7 on the same draw (notebook §7.4): what the battery sold vs. what it can deliver, the SOC paths (plan, day-ahead, real time, benchmark), the generator redispatch, and the real-time dispatch.



4.9 Welfare and metrics

$$SW_{\text{real}} = -\Big(\Delta T \sum_{j,t} c_j q^{rt}_{jt} + \psi\, \Delta T \sum_{j,t} |q^{rt}_{jt} - q^{da}_{jt}| + \text{VOLL}\, \Delta T \sum_{t} sh_t + \Delta T \sum_{e,t} (\varepsilon_e p^{rt}_{et} + \zeta_e g^{rt}_{et}) - \sum_{e} w_e s^{rt}_{eT}\Big)$$

$$SW_{\text{noESS}} = -f^{\star}\big(\mathcal{P}^{\text{SOC}} \text{ with no battery + VOLL generator}\big), \qquad V = SW^{\star} - SW_{\text{noESS}} \quad \text{(storage value)}$$

$$\Delta SW = SW^{\star} - SW_{\text{real}} \quad (\text{asserted} \geq -10^{-6}), \qquad \Delta SW\% = 100\, \Delta SW / V$$

Further metrics (`metrics.py`): price distortion $\overline{|\lambda^{\text{CD}} - \lambda^{\star}|}$; decomposition quantity part $= \Delta SW$(`B_star_p`), price part $= \Delta SW$(`B_fcst`) $- \Delta SW$(`B_star_p`) (accounting split, not causal); $\sigma^{\star}$ where mean loss of `B_fcst` crosses that of `B_priv`; mean, 95% CI, median, p95 per setting. Battery profit is the day-ahead settlement at λ^{CD} minus degradation (real-time costs it causes are not charged to it; flagged assumption).

5 The forecast error

5.1 Model (`forecasts.forecast_prices`)

For battery e and interval t, with $\bar{\lambda} = \text{mean}(\lambda^{\star})$:

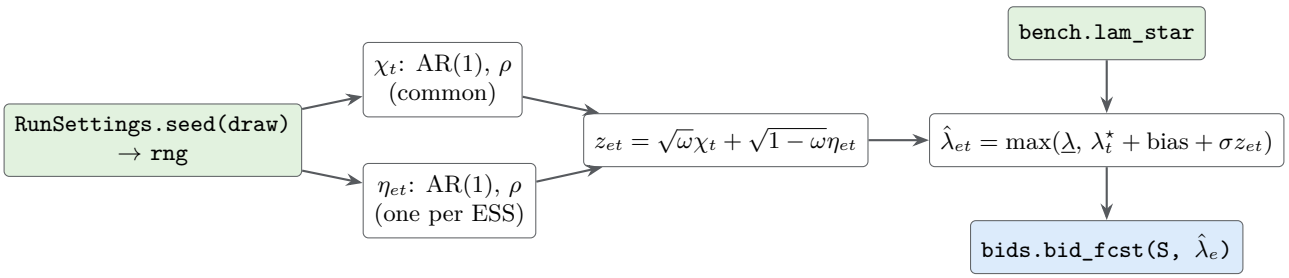
$$\hat{\lambda}_{et} = \max\Big(\underline{\lambda},\ \lambda^{\star}_t + \underbrace{\tfrac{\texttt{bias_pct}}{100}\,\bar{\lambda}}_{\text{bias}} + \sigma\, z_{et}\Big), \qquad z_{et} = \sqrt{\omega}\,\chi_t + \sqrt{1-\omega}\,\eta_{et},$$

$$\sigma = \begin{cases} \texttt{sigma_abs} & \text{if given (\$/MWh)} \\ \frac{\texttt{sigma_pct}}{100}\,\bar{\lambda} & \text{otherwise} \end{cases}$$

χ (common to all batteries) and η_e (battery-specific) are independent unit-variance AR(1) paths, each drawn by `ar1_unit_variance`:

$$x_1 \sim \mathcal{N}(0,1), \qquad x_t = \rho\, x_{t-1} + \sqrt{1-\rho^2}\, w_t, \quad w_t \sim \mathcal{N}(0,1) \text{ i.i.d.},$$

hence $\text{Var}\, z = 1$, $\text{corr}(z_t, z_{t+1}) = \rho$ and $\text{corr}(z_e, z_{e'}) = \omega$. With $\sigma = 0$ the function returns $\max(\underline{\lambda}, \lambda^{\star} + \text{bias})$ exactly, without touching the RNG. The **price floor** $\underline{\lambda}$ (`ErrorModel.price_floor`, default 0, `None` disables it) keeps the forecast inside the range of prices the market can actually produce (§6).



5.2 Seeds: one error shape per draw, shared by every setting

To compare error sizes (and, next, bid designs) properly, every setting must face the *same* forecast-error scenarios. A difference in loss between $\sigma = 1$ and $\sigma = 5$ should come from the size of the error alone, not from one setting happening to draw more harmful error shapes than the other. We therefore derive the seed of draw d from the draw number only,

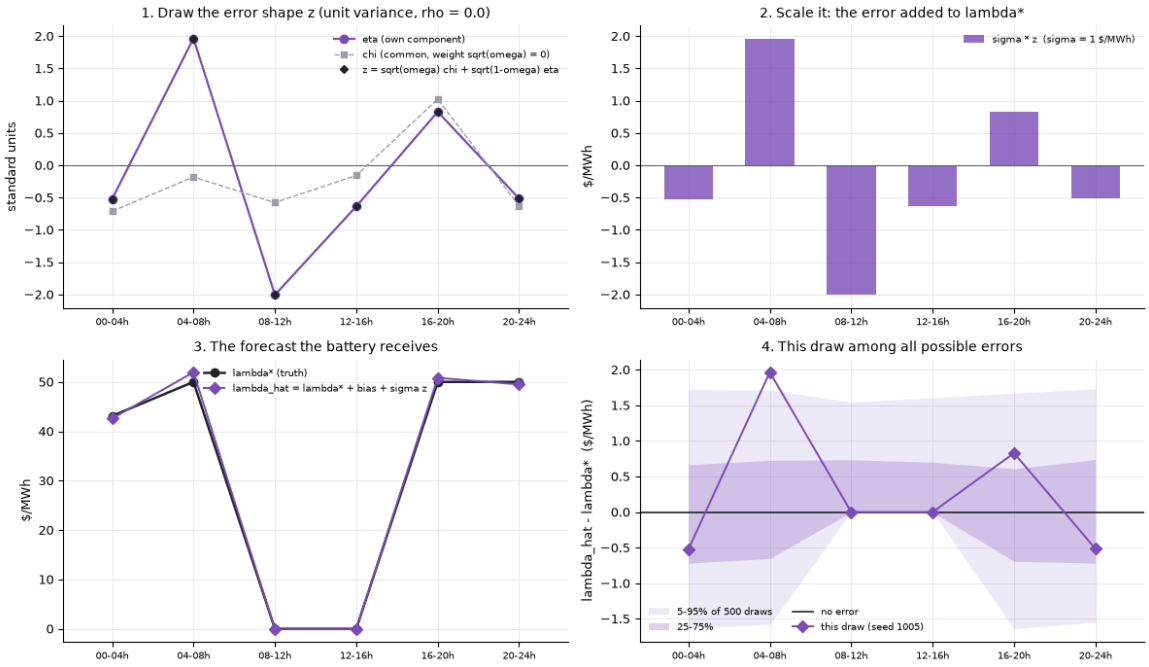
$$\texttt{seed}(d) = \texttt{SeedSequence}(\texttt{base_seed}, d),$$

and all randomness of the draw comes from one generator, `rng = default_rng(RunSettings.seed(d))`. Because the seed does not depend on the setting, draw d uses the same innovations w_t, and so the same χ, η and z, in every setting that shares `base_seed`. σ and the bias only scale and shift that shape, and ρ and ω only change how the same innovations are combined. This is the **common random numbers** (CRN) design:

	$\sigma = 1$	$\sigma = 5$	$\sigma = 20$
draw d	shape A $\times 1$	shape A $\times 5$	shape A $\times 20$
draw $d+1$	shape B $\times 1$	shape B $\times 5$	shape B $\times 20$

A comparison between two settings is then *paired*: both face the same forecast shapes, and the difference comes from the parameter or the design alone. This also makes the comparison much more precise: $\text{Var}(\bar{L}_A - \bar{L}_B) = \text{Var}\,\bar{L}_A + \text{Var}\,\bar{L}_B - 2\,\text{Cov}(\bar{L}_A, \bar{L}_B)$, and shared shapes make the covariance large. Since the settings are not independent samples, the confidence interval of a *difference* is computed from the per-draw differences (`metrics.paired_difference`), not from the two settings' own intervals. Finally, `SeedSequence` is a fixed function of (`base_seed`, d), so every draw can be regenerated in any session from its number; each result row stores its seed. Stage-0 effect: §12.

The same steps on one draw (notebook §3, seed 1005, $\sigma = 1$ \$/MWh, floor 0): the error shape z, the scaled error, the resulting forecast, and where this draw sits among 500 draws. In the two zero-price intervals the floor clips the forecast to 0.



5.3 Where the error is inserted

Only in `runner.run_draw`, branch `family == "B_fcst"`: $\hat{\lambda}_e$ replaces $\lambda^\star$ as the input of M3 (self-schedule) *and* M4 (polyhedron). It therefore corrupts both parts of the bid:

- the **quantity** (p°, g°): the battery plans to charge/discharge in the wrong hours;
- the **price** (u, c): the multiplier π (value of stored energy) is computed at the wrong prices.

Nothing downstream sees $\hat{\lambda}$: clearing (M5), repair (M6), redispatch (M7) and welfare use the true system and true battery parameters. The error is a *price* error, not a renewable or load error: the system itself is deterministic.

6 Choosing the error values, and how to defend them

The error values are a **design of experiments, not an estimate**. Each one is chosen by four rules that can be stated and defended independently of the results:

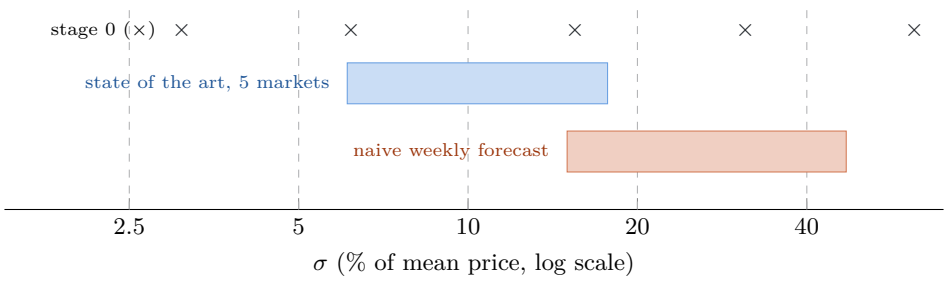
1. **Scale-free units.** σ and the bias are percentages of the mean benchmark price $\bar{\lambda}$, so a result transfers across systems and currencies, and matches how forecast accuracy is reported in the literature (relative errors).
2. **Bracket reality, include the control.** Each axis runs from *perfect information* (0, the control: it must reproduce the zero-error result) to *a naive forecaster*, with the published state-of-the-art values in between.
3. **Geometric spacing.** Doubling steps resolve small errors finely and still reach poor forecasts with few settings.
4. **Calibration replaces stylised values.** Stage 3 (`experiments.md`, "Calibration") backtests a naive forecaster on real DK1/DK2 prices and estimates σ, bias, ρ and ω. The grid's job is to show the *shape* of the loss; the calibrated point is then placed on it.

6.1 Forecast error size σ: {0, 2.5, 5, 10, 20, 40} % of $\bar{\lambda}$

Anchor. The open benchmark of Lago et al. (2021) evaluates state-of-the-art day-ahead price forecasters on five markets with two-year test sets. Best model (ensemble of neural networks), their Table 3:

	Nord Pool	PJM	EPEX-BE	EPEX-FR	EPEX-DE
sMAPE (%)	4.85	11.22	13.33	10.98	14.19
rMAE (MAE relative to a naive weekly forecast)	0.403	0.439	0.573	0.533	0.377
implied σ, state of the art (%)	6.1	14.0	16.7	13.7	17.7
implied σ, naive forecast (%)	15.0	31.9	29.1	25.7	47.0

Conversion: for a Gaussian error, $\mathrm{MAE} = \sigma\sqrt{2/\pi} \approx 0.80\,\sigma$, so $\sigma \approx 1.25\,\mathrm{MAE}$; the relative MAE is approximated by the sMAPE (an approximation, flagged as such), and the naive forecaster's error is the best model's divided by its rMAE. Mapped onto the grid (dashed lines: stage-1 values; ×: stage-0 values $\{1, 2, 5, 10, 20\}$ \$/MWh $= \{3.1, 6.2, 15.5, 31, 62\}$% of $\bar{\lambda} = 32.18$):



Defence. 2.5% is better than any published model (it tests the small-error limit and continuity at 0); 5 and 10% are best-in-class markets; 20% is a typical market or a weaker model; 40% is a naive forecaster. The grid therefore spans from perfect information to no forecasting skill, with every published value inside it. Precedent: Martinez-Rico et al. (2020), in a battery-bidding sensitivity analysis, use error standard deviations of 0, 5, 10,

15 and 20% (on renewable production, not price). The stage-0 values in $/MWh are inherited from the pilot so its results can be compared; they cover the same range.

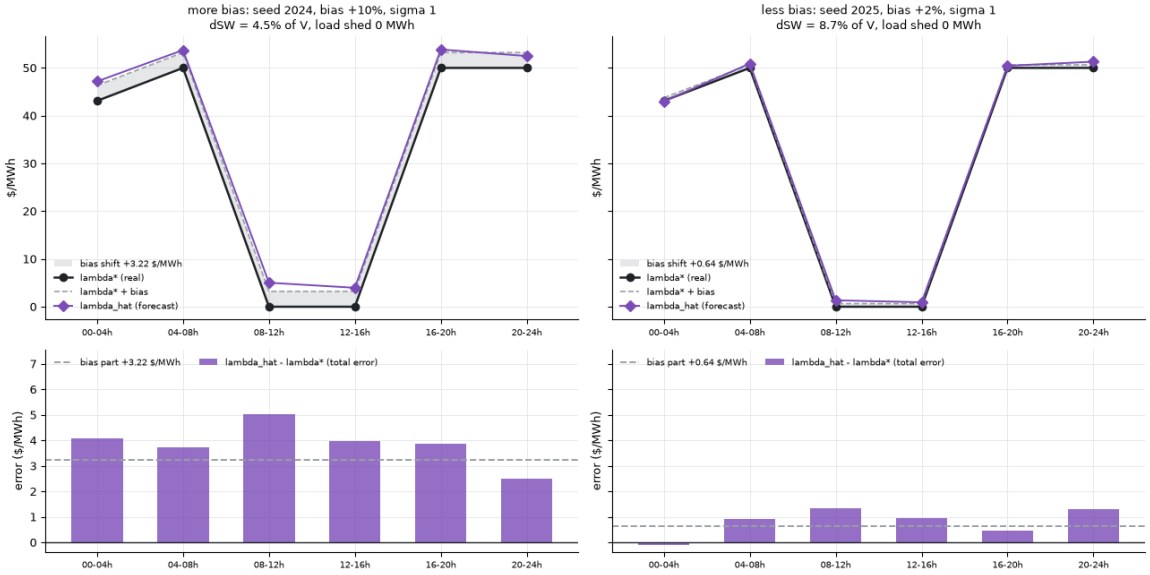
Caveat to state. Real price errors are heavier-tailed than Gaussian: Lago et al.'s RMSE/MAE ratios are 1.7–3.0, against 1.25 for a normal distribution. The Gaussian model is the maximum-entropy choice for a given mean and variance and isolates the effect of size; the stage-3 bootstrap of real residuals (`forecasts.bootstrap_days`) is what captures spikes.

6.2 Autocorrelation ρ: {0, 0.5, 0.9} between consecutive intervals

Anchor (own data). Price forecast errors inherit the persistence of their drivers, above all the wind forecast error. In the RTS-GMLC data of this project (`Data/timeseries_hourly_2020.csv`), the day-ahead wind forecast error has autocorrelation 0.87 at 1 h, 0.71 at 2 h, **0.49 at 4 h** and 0.23 at 8 h; for PV, 0.73 / 0.49 / 0.22 / −0.07. An interval of the test case lasts 4 h, so $\rho \approx 0.5$ is the data-anchored central value. $\rho = 0$ (independent errors) is the lower bound; $\rho = 0.9$ is the hourly persistence of wind errors, i.e. the right value if the model is moved to hourly resolution, or a forecast that is off by a near-constant amount all day. *If ΔT changes, the grid must be re-mapped*: the relevant quantity is the error's autocorrelation at lag ΔT.

6.3 Bias: {−10, 0, +10} % of $\bar{\lambda}$

A bias is a systematic level error: fuel or CO_2 prices moving after the model was trained, a regime the training data did not contain. ±10% is the same size as the state-of-the-art random error (6–18%), so the grid compares a systematic and a random error of equal magnitude; it is symmetric so that sign effects can show up. One is built in: with the price floor, a negative bias is cut off in the zero-price intervals (the forecast cannot go below 0), a positive one is not. The real bias is the mean residual of the stage-3 backtest. Two draws, one with more and one with less bias (notebook §3); bottom row: the error split into its bias (dashed) and random part:



6.4 Common share ω: {0, 0.5, 1} (stage 2, several batteries)

$\omega = 0$: each battery has its own forecaster, errors independent; $\omega = 1$: all use the same forecast (a common vendor or a public TSO forecast), so their errors move together; 0.5 the midpoint. These are the two logical extremes; stage 3 estimates ω as the correlation between the residuals of two different forecasters.

6.5 Price floor $\underline{\lambda} = 0$

In the test system the cheapest supply is free, curtailable renewables and nobody bids below 0, so $\lambda^\star_t \geq 0$ always (here $\lambda^\star \in [0, 50]$). A forecast below 0 predicts a price this market cannot produce; a forecaster that knows the market's price range would not make it. The floor changes the error distribution only where the true price is near 0: with $\lambda^\star_t = 0$, the error becomes $\max(0, \sigma z)$, zero half the time and $\sigma/\sqrt{2\pi} \approx 0.40\,\sigma$ on average (an upward bias in those intervals). **It matters a lot here:** without it, about 1 draw in 5 is catastrophic at every σ (Finding 2 in §7). Real markets can clear below zero (DK1 had 281 negative-price hours in 2023,

Forsyningstilsynet), driven by subsidies and inflexible generation; for a realistic stage-3 system the floor must be set to that market's own minimum, or disabled. That is why it is a parameter, not a constant.

6.6 Not error parameters, but chosen alongside them

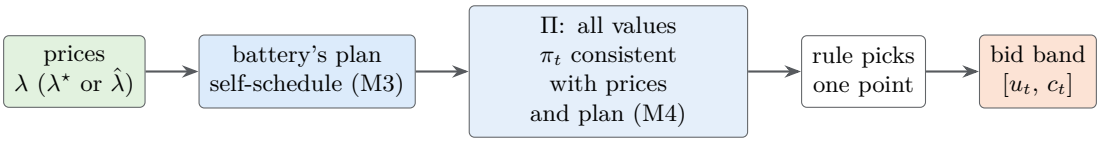
- **Real-time premium ψ: {0, 10, 50} \$/MWh.** 0 is frictionless correction (lower bound on the loss). Martinez-Rico et al. (2020) apply imbalance penalties of 13–14% of the price in the Iberian market, about 4.5 \$/MWh at $\bar{\lambda} = 32$; $\psi = 10$ ($\approx 31\%$ of $\bar{\lambda}$) is a conservative version of that, $\psi = 50$ ($\approx 155\%$) a scarcity stress test. Still an open decision with the supervisors (`experiments.md` §8).
- **Draws N.** With the floor, the standard deviation of $\Delta SW\%$ within a setting is 3–5 points (stage 0). The 95% confidence half-width of a mean, $1.96\, s/\sqrt{N}$, is 0.5–0.8 points at $N = 150$ and 0.3–0.4 at $N = 500$ (stage 1): small enough to rank neighbouring grid points.

Sources. J. Lago, G. Marcjasz, B. De Schutter, R. Weron, "Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark", *Applied Energy* 293 (2021) 116983, Table 3. J. Martinez-Rico et al., "Forecast error sensitivity analysis for bidding in electricity markets with a hybrid renewable plant using a battery energy storage system", *Sustainability* 12 (2020) 3577. Forsyningstilsynet, *The Danish Electricity and Natural Gas Markets 2023*. RTS-GMLC time series, `Data/timeseries_hourly_2020.csv` (autocorrelations computed for this document).

7 Inside the multiplier polyhedron Π

7.1 What Π is, in plain words

To bid, the battery needs one number per interval: π_t, **the value of one stored MWh at the end of interval** t. The bid follows from it (formula (E)): it will *buy* at prices up to $u_t = \alpha(\pi_t - \bar{\nu}_t) - \varepsilon$ (what a stored MWh is worth, minus losses) and *sell* at prices from $c_t = \zeta + \beta(\pi_t + \underline{\nu}_t)$ (what it gives up by selling it, plus losses). Π is the set of all π (with the matching ν) that are consistent with the prices and with the battery's own planned schedule. Every point of Π is a valid bid; the rule (`prelimit` or `solver`) picks one.



Why a set, not a single value. Each π_t is squeezed between two bounds, each set by a price the battery reacted to in its plan: a stored MWh is worth *at least* what it can still earn later (the price at which the battery would sell it), and *at most* what the battery passed up (the price at which it chose not to buy more, or chose to sell rather than keep). Any value in between is consistent. If the two prices give different numbers, π_t has a range; if they give the same number, it is pinned. The state of charge decides which prices bound which interval: π can only change from one interval to the next when the battery hits empty or full in between.

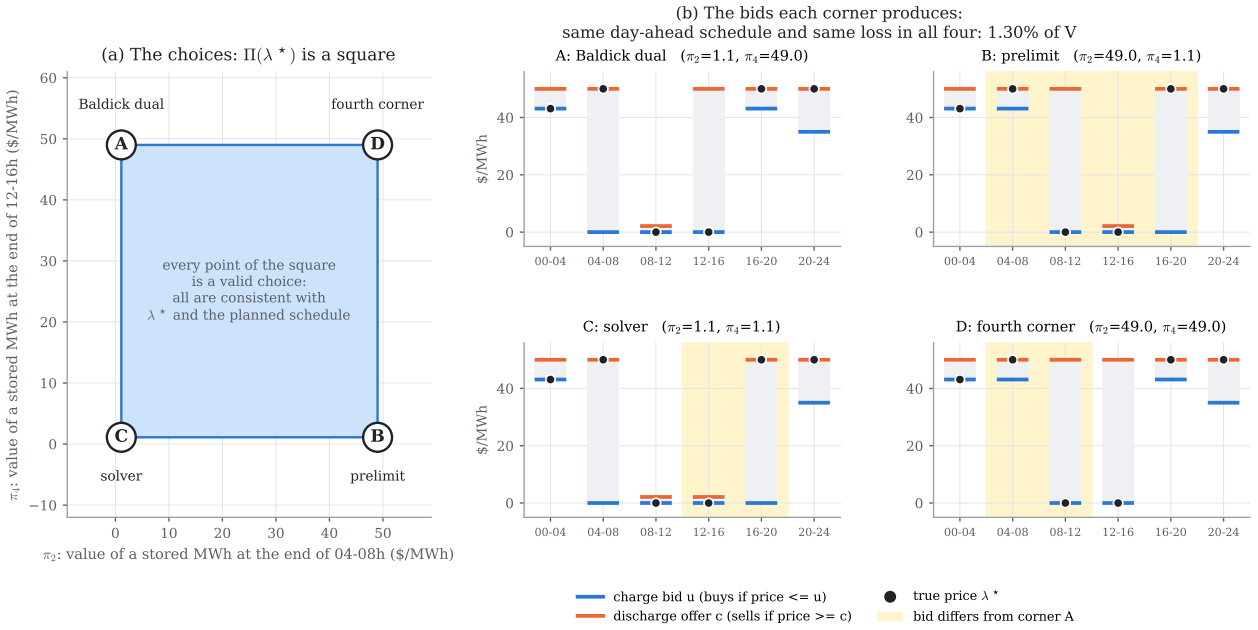
In the test system, at $\lambda^\star$: π_2 and π_4 lie anywhere in [1.11, 49] \$/MWh, between refilling with free energy at 08–16h ($(0+\varepsilon)/\alpha = 1.11$) and selling at 16–24h ($(50-\zeta)/\beta = 49$). The others are pinned *by ties* in $\lambda^\star$: π_5 is bounded by the prices at 16–20h and 20–24h, both 50; π_1 by buying at 00–04h, $(43.1+\varepsilon)/\alpha = 49$, and selling at 04–08h, $50 - \zeta = 49$ (a break-even round trip); π_3 by the two free intervals, both priced 0.

How the pictures are made. Π lives in $\mathbb{R}^{3T} = \mathbb{R}^{18}$ and is unbounded in the ν directions (at an hour where the plan does not charge, any less aggressive charge bid stays consistent). Its π part is bounded and is the part that matters, so the pictures show π. They are exact: each point is an LP optimum over Π, and welfare is computed through `runner.evaluate_blocks`, the same pipeline as the experiments, at $\psi = 10$ (`scripts/plot_multiplier_polyhedron.py`).

7.2 At the true prices: a square of choices, all equally good

The battery's self-schedule at $\lambda^\star$ is *not* Baldick's schedule: both are optimal for the battery (Lemma 4), because $\lambda^\star$ makes it indifferent between them. The tie-breaker of M3 (§4.3) picks the most active of the tied plans: charge 222 MW at 00–04h and at 12–16h, discharge 200 MW at 04–08h and at 20–24h. That plan leaves the battery empty at the end of 04–08h and full at the end of 12–16h, so exactly two values are free, π_2 and π_4, each anywhere in [1.11, 49]; all others are pinned ($\pi_1 = 49$, $\pi_3 = 1.11$, $\pi_5 = 49$, $\pi_6 = w = 40$). $\Pi(\lambda^\star)$ is therefore a

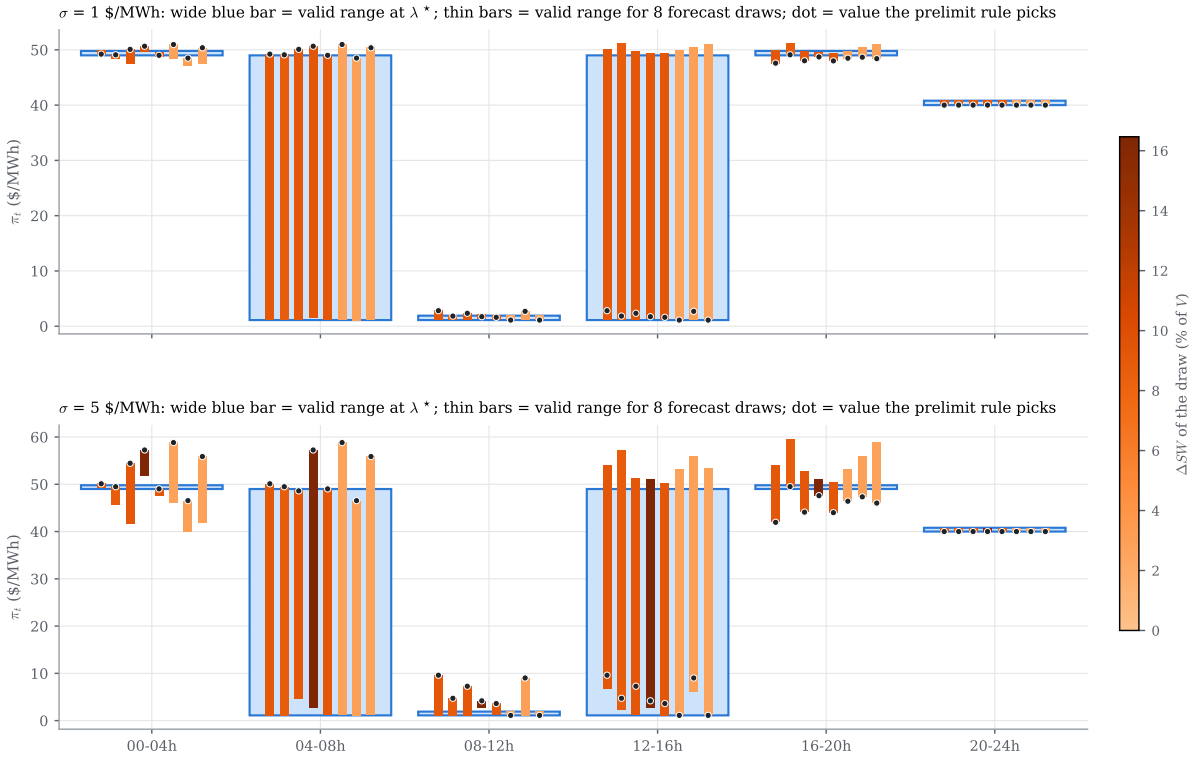
square. Its corners include Baldick's own dual (A) and the two rules (B, C), and each corner produces visibly different bids (yellow), for example a charge bid of 43.1 or 0 \$/MWh at 04–08h:



Finding 1. The four corners give *exactly the same* day-ahead schedule (charge 100 MW at 00–04h and 200 MW at 12–16h; discharge 100 MW at 04–08h, 75 MW at 16–20h and 100 MW at 20–24h), the same prices and the same loss, 1.30% of V. In every interval where their bids differ, the market price stays on the same side of the bid, so the accepted quantities do not change. At $\lambda^\star$ the choice of rule is therefore irrelevant: the whole `B_star_p` loss comes from the *quantity plan* (which of the tied optimal schedules the battery plans), not from the bid prices. Baldick's dual with Baldick's own quantities (`B_star`) gives 0.

7.3 Under forecast error: the valid ranges move and open up

A forecast breaks the ties of $\lambda^\star$ (prices of exactly 0 at 08–16h, a break-even round trip at 00–08h), so the battery's plan changes and with it the set of valid values. The clearest view is one interval at a time: the range each π_t can take, at the true prices and for 8 forecast draws:



How to read it: at $\lambda^\star$ (wide blue bars) only 04–08h and 12–16h have freedom. Under a forecast the ranges move, and the value the prelimit rule picks (dots) lands in different places, so the bids differ from draw to draw; the colour of each draw is its loss. Intervals that were pinned open up (00–04h, 08–12h and 16–20h), because they were pinned only by ties in $\lambda^\star$: the two prices that bound them were exactly equal (§7.1). A forecast puts slightly different numbers on them, e.g. 52 at 16–20h and 48 at 20–24h instead of 50 and 50, and the bounds separate by about that difference: $\pi_5 \in [47, 51]$ instead of $\{49\}$. That is why the opened ranges are narrow at $\sigma = 1$ and wider at $\sigma = 5$. In full dimension, $\Pi(\hat{\lambda})$ is 5-dimensional in 12 of these 16 draws and 4-dimensional in the 4 where the floor clips both zero-price intervals to exactly 0, which restores one tie (exact 2-D and 3-D shadows of it: `fig_poly_forecast.pdf` and `fig_poly_3d.pdf` in `docs/figures/`).

Per-draw $\Delta SW\%$ of the prelimit bid (price floor 0): $\sigma = 1$: 9.3, 8.7, 9.3, 9.3, 9.3, 2.8, 3.1, 2.8; $\sigma = 5$: 9.3, 8.7, 9.3, 16.5, 9.3, 2.8, 3.1, 2.8. No draw sheds load.

Finding 2: why the forecast is floored at 0. Prices in this market cannot be negative (the cheapest supply, curtailable renewables, is offered at 0), so the forecast is not allowed to be either: $\hat{\lambda}_t \geq 0$ (`ErrorModel.price_floor = 0`). The floor matters because of the free intervals $t = 3, 4$, where the true price is exactly 0. A charge block is accepted there only if its bid is ≥ 0, and the bid follows the forecast ($u_t = \hat{\lambda}_t$ where the battery plans to charge). A negative forecast in both intervals would make the battery buy nothing, reach the 16–20h shortfall empty, and force real time to shed 75 MW at VOLL (mechanism traced step by step in the stage-0 notebook, Section 7). Because $\lambda^\star = 0$ exactly there, an unbounded error would be negative with probability 1/2 whatever σ is, so this unphysical event would dominate every result. With the floor, the battery always buys the free energy and the loss grows with σ. 200 seeds per σ (seeds 1000–1199, prelimit, $\psi = 10$):

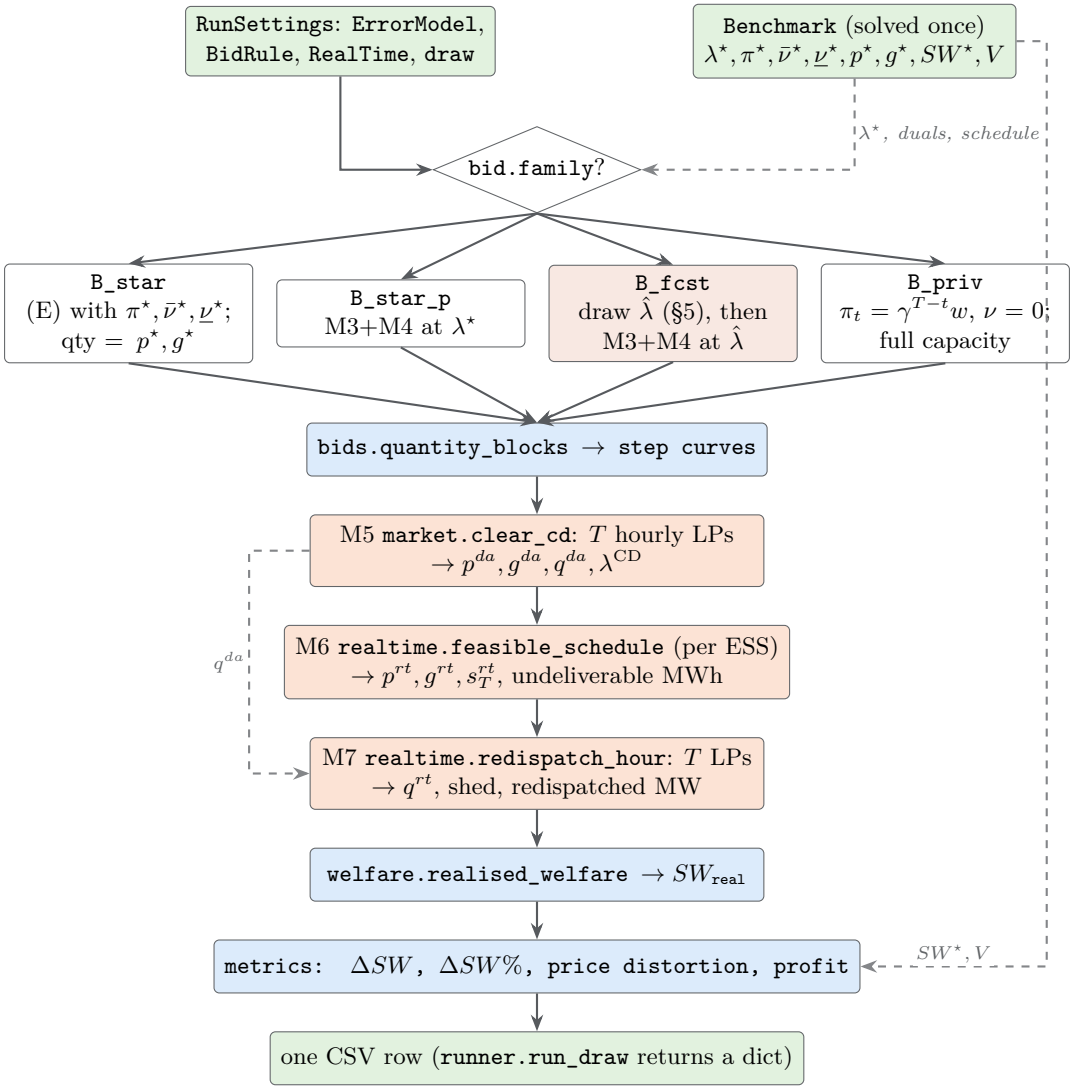
σ ($/MWh)	1	2	5	10	20
mean $\Delta SW\%$	6.3	6.3	6.8	8.1	9.0
draws with load shed	0	0	0	0	0

The tie-breaker of M3 never changes the plan in these 1 000 draws. Load shedding is rare but not impossible: none of seeds 1000–2999 at $\sigma = 20$ sheds, but it can happen when the floor clips several intervals at once, e.g. for the forecast $\hat{\lambda} = (0, 0, 1.2, 0, 32, 48.3)$. There the floor clips three intervals to exactly 0, the battery is indifferent *when* to charge, and it plans to buy at 00–04h (true price 43.1), where the market lets it take only 100 MW. It runs empty by 16–20h and 75 MW is shed. This is a timing tie, which throughput cannot break (§4.3).

The remaining sensitivity to near-ties in this small case (losses jump between a few discrete values, 2.8, 3.1, 8.7,

9.3, 16.5 %) is a property of the test system, not of the design. The tie-breaker removes the *exact* ties of the $\sigma = 0$ plan, but a forecast error turns them into *near*-ties that the error itself decides, in a random direction. A case without near-ties, or the block-cost perturbation of `CLAUDE.md` §7, should make the loss grow continuously with σ.

8 One experiment draw, end to end



Around the draw (`runner.run_all`): for each `RunSettings` in the list from `config`, skip if `results/<stage>/<setting_id>.csv` exists (resume), else call `run_setting` (= `run_draw` for `draw`= 0..`draws`−1), write the CSV, and finally `meta.json` (date, git commit, versions, $SW^\star$, SW_{noESS}, V, $\lambda^\star$). Inside `run_draw`, everything from day-ahead clearing to welfare (M5–M7 and welfare) is one function, `runner.evaluate_blocks`, which takes the case, the bid blocks and the real-time settings and returns an `Outcome` with every intermediate result. Any bid curve can therefore be scored through the identical pipeline, and inspected step by step (§9).

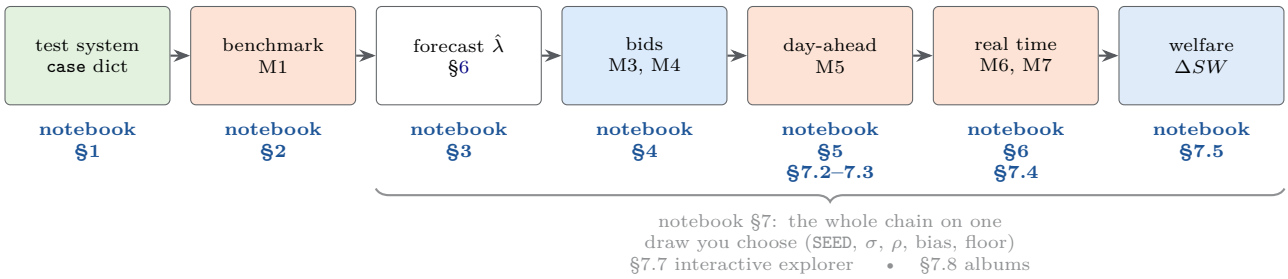
Cost per draw (`B_fcst`, one battery, $T = 6$): 1 (M3) + 12 (`pi_range`, M4) + 6 (M5) + 1 (M6) + 6 (M7) = 26 small LPs with `"prelimit"`; 15 with `"solver"` (one vertex LP instead of `pi_range`).

8.1 What each bid family isolates

Family	Prices from	Quantity plan	Role	Expected ΔSW
`B_star`	$\lambda^\star$, Baldick's own duals	benchmark $(p^\star, g^\star)$	gate: equivalence holds	0
`B_star_p`	$\lambda^\star$, M4 selection	own self-schedule at $\lambda^\star$	cost of selecting from Π	≥ 0
`B_fcst`	$\hat{\lambda}_e$, M4 selection	own self-schedule at $\hat{\lambda}_e$	**main experiment**	$\geq$ `B_star_p`
`B_priv`	none: $\pi_t = \gamma^{T-t} w$	full capacity	lower bound (technical data only)	reference

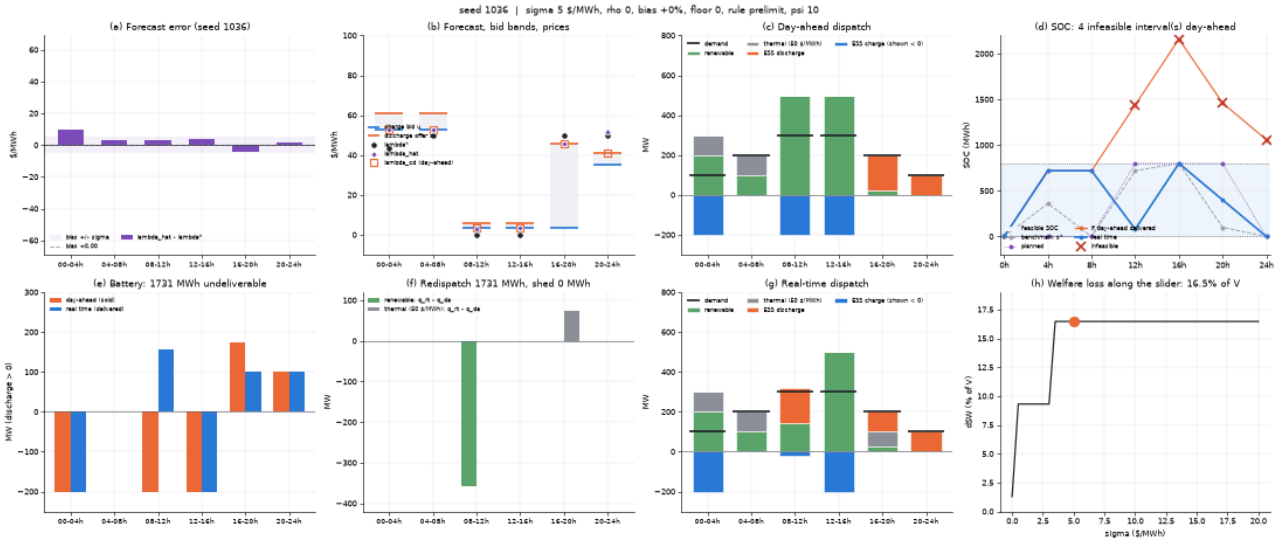
9 Seeing each step: notebook, explorer and albums

Every box of the pipeline above has a picture in `notebooks/stage0_experiments.ipynb`. The notebook follows the model's order, so it can be read side by side with this document:



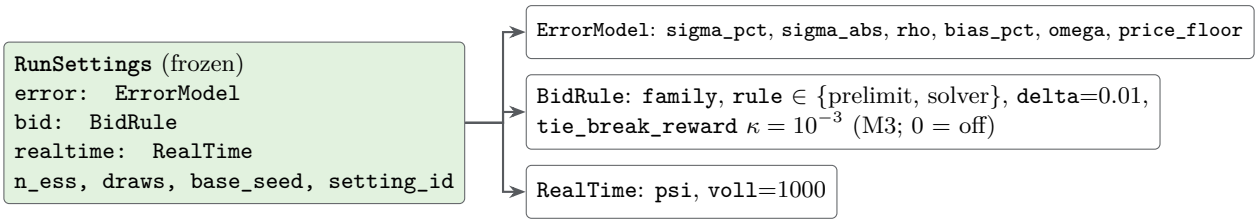
Interactive explorer (notebook §7.7). Widgets for the seed, σ, ρ, bias, price floor, bid rule and ψ; every change re-solves one draw (about 0.03 s) and redraws eight panels: (a) the forecast error, (b) forecast, bid bands and prices, (c) day-ahead dispatch, (d) SOC feasibility, (e) sold vs. delivered, (f) redispatch and shedding, (g) real-time dispatch, (h) where the loss comes from, cost by cost. A status line says whether the last change moved the dispatch; a play control steps one parameter through its range.

Albums (notebook §7.8, and the standalone page `docs/figures/album_seed1036.html`). For one error shape z (one seed), one album per parameter (σ, ρ, bias, ψ, floor, rule), each picture rendered in advance and browsed with a labelled slider; panel (h) becomes the loss along the slider. One picture of the σ album:



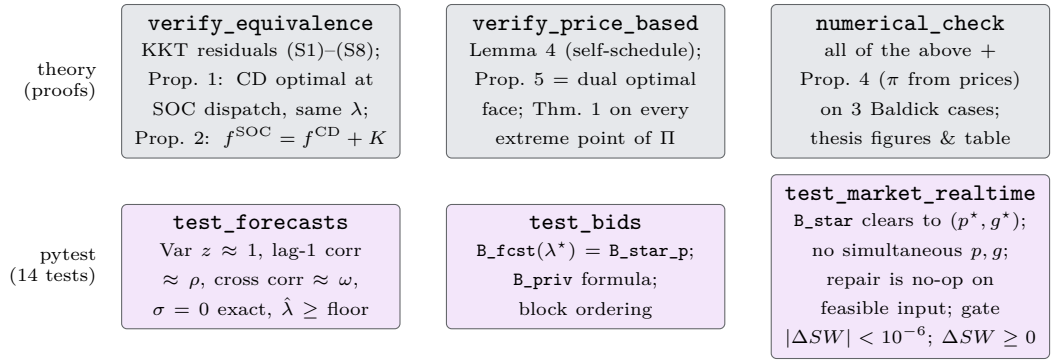
What to take from it: with z fixed, $\hat{\lambda}$ moves *continuously* with σ, ρ and the bias, but the market outcome moves in *steps*, because the battery's schedule only changes when the forecast crosses a decision threshold. At $\sigma = 1$ \$/MWh, 40 seeds give only 8 distinct day-ahead battery schedules (15 at $\sigma \geq 5$); a σ sweep at a fixed seed crosses at most two thresholds in this test system.

10 Configuration and stages (`config.py`)

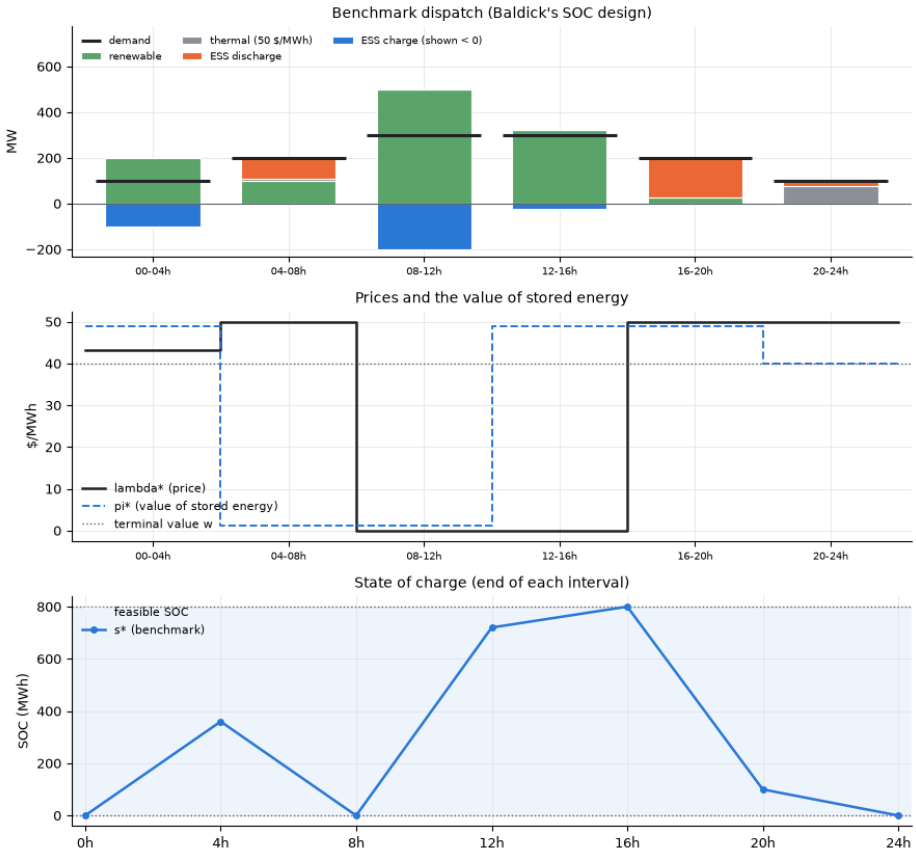


Stage	Grid	Status
0 (gate)	`B_star`; `B_priv` at $\psi \in \{0, 10, 50\}$; `B_star_p`, `B_fcst` at $\sigma \in \{0, 1, 2, 5, 10, 20\}$ \$/MWh, $\rho = 0$, $\psi = 10$, prelimit, 150 draws	16 settings, 1 506 draws: **run**
1	`B_fcst`: $\sigma\% \in \{0, 2.5, 5, 10, 20, 40\} \times \rho \in \{0, 0.5, 0.9\} \times$ bias $\in \{-10, 0, 10\}\% \times$ rule $\times$ $\psi \in \{0, 10, 50\}$ = 324 settings, 500 draws	defined, not run
2	fleet of 5–20 batteries, 10–40% of peak; $\rho \in \{0, 0.9\}$, $\omega \in \{0, 0.5, 1\}$	blocked: `fleet_case` raises
3	realistic system + calibrated/empirical errors	`bootstrap_days` stub

11 Verification layer



12 Numbers: benchmark and stage 0



Baldick's benchmark (notebook §2): dispatch, prices $\lambda^\star$ against the value of stored energy $\pi^\star$, and the SOC path.

t	1	2	3	4	5	6
$\lambda_t^\star$	43.1	50	0	0	50	50
$s_t^\star$	360	0	720	800	100	0
$\pi_t^\star$	49	1.1	1.1	49	49	40
$\bar{\nu}_t^\star$	0	0	0	47.9	0	0
$\underline{\nu}_t^\star$	0	47.9	0	0	0	9

$SW^\star = -19\,449$, $SW_{\text{noESS}} = -360\,000$, $V = 340\,551$ \$/day.

V is dominated by reliability: without the battery, interval 5 has a 75 MW shortfall (25 MW renewable + 100 MW thermal vs. 200 MW demand) shed at VOLL for 4 h (= 300 000 \$). $\lambda_1^\star = 43.1 = a\,\pi_1 - \varepsilon$: the battery is the marginal unit there.

Setting (stage 0, price floor 0)	draws	mean ΔSW%	sd	median	p95	undeliverable MWh
`B_star`	1	0.00		0.00	0.00	0
`B_star_p` (any σ)	1/150	1.30	0	1.30	1.30	40
`B_fcst`, $\sigma = 0$	1	1.30		1.30	1.30	40
`B_fcst`, $\sigma = 1$	150	5.80	3.10	4.47	9.31	976
`B_fcst`, $\sigma = 2$	150	5.80	3.10	4.47	9.31	976
`B_fcst`, $\sigma = 5$	150	6.22	3.68	4.47	9.31	1 002
`B_fcst`, $\sigma = 10$	150	7.11	4.64	7.49	16.46	1 050
`B_fcst`, $\sigma = 20$	150	7.85	4.88	8.22	16.46	1 091
`B_priv`, $\psi = 0/10/50$	1	0.00 / 3.09 / 15.43				1 051

Reading: the gate passes (`B_star` gives $\Delta SW = 0$). At zero error, building the bid from the tie-broken self-schedule and a selected point of Π costs 1.30% of V (§4.3, Finding 1). As soon as there is any error the mean jumps to about 6%, and it rises with σ to 7.9% at 20 \$/MWh. No draw of this run sheds load. The jump from 1.3 to 6% is the cost of *near*-indifference: at $\sigma = 0$ the tie-breaker picks a good plan, but a forecast error of 1 \$/MWh already makes the battery strictly prefer one of the tied plans, chosen by the sign of the error. The tie-breaker changes the plan in none of the 750 forecast draws: under forecast error there are no exact ties left for it to break. `B_priv` at $\psi = 10$ (3.1%) still beats `B_fcst` on average here.

Paired comparison (common random numbers, §5.2). All rows use the same 150 error shapes, so neighbouring error sizes can be compared draw by draw (`metrics.paired_difference`; difference = loss at the larger σ minus loss at the smaller):

step in σ (\$/MWh)	mean difference	95% CI	half-width, paired	half-width, unpaired	draws not lower
$1 \to 2$	0.00	[0.00, 0.00]	0.00	0.70	100%
$2 \to 5$	0.42	[0.13, 0.70]	0.29	0.77	99%
$5 \to 10$	0.90	[0.53, 1.27]	0.37	0.95	100%
$10 \to 20$	0.74	[0.39, 1.09]	0.35	1.08	99%

The paired intervals are 2.5–3 times narrower, so every step from $\sigma = 2$ upwards is resolved: a larger error costs more. In 146 of the 150 draws the loss never decreases as σ grows. In 99 of them it is the *same* at every σ: the *shape* of the error decides the battery's plan, and its size rarely does. That is also why $\sigma = 1$ and 2 give identical losses in every draw. These are the numbers in `results/stage0/*.csv`; the notebook reproduces them exactly, in any session.

13 Notation crosswalk

Meaning	Code (`case["ess"]`)	Thesis / `experiments.md`	Root `CLAUDE.md`
charge efficiency (energy in per MWh bought)	`a`	α	α
discharge factor (energy out per MWh sold)	`b`	β	$1/\beta$ (!)
self-discharge retention	`g`	γ	–
charge / discharge cost	`eps`, `zet`	ε, ζ	ε, ζ
charge / discharge power	`p`, `g` (LP vars)	p, g	c, d
SOC limits, initial SOC	`smin`, `smax`, `s0`	$\underline{s}$, $\bar{s}$, s_0	$\bar{S}$, s_0
power limits	`pbar`, `gbar`	$\bar{p}$, $\bar{g}$	$\bar{C}$, $\bar{D}$
terminal value (linear)	`w`	w, $b(s_T) = ws_T$	$b(s_T)$
SOC dual (value of stored energy)	`y["pi",e,t]`	π_t	μ_t
SOC-limit duals	`nuU`, `nuL`	$\bar{\nu}$, $\underline{\nu}$	–
charge bid / discharge offer	`u`, `c`	u, c	π^{bid}, π^{offer}

The root `CLAUDE.md` brief (RTS-GMLC data, `dp.py`, `curves.py`, milestones M1–M6) describes the earlier framework now in `Archive/guide_md_framework/`; it is *not* the code documented here. In particular its β divides discharge, whereas the code's `b` multiplies it.

14 Caveats found while reading the code

1. **Common error component breaks with several batteries.** `run_draw` calls `forecast_prices(..., n_ess, ...)` *inside* the per-battery loop and keeps row `e`, so χ is redrawn for every battery and the cross-battery correlation is no longer ω. Harmless for $n_{\text{ess}} = 1$ (stages 0–1); must be fixed before stage 2 (draw once, before the loop).
2. **`B_star_p` ignores** σ, so the five stochastic `B_star_p` settings in stage 0 are 750 identical draws (all 1.30%).
3. **Hourly clearing assumes no binding ramps.** `market.clear_cd` solves each hour alone; a system with binding ramps or a network needs a multi-hour version.
4. **Test system is the substitute `baldick_case`**, not the pilot `day_case` (`pilot/` is absent), so the published pilot numbers cannot be matched, and V mostly reflects avoided shedding.